

PE 文件导出表结构与 ROR13 哈希算法

● 为什么不能用 GetProcAddress

之前写注入器时，一定写过这样的代码：

```
HMODULE hKernel32 = GetModuleHandleA("kernel32.dll");
FARPROC funcAddr = GetProcAddress(hKernel32, "CreateRemoteThread");
```

- 虽很简洁但有个缺陷：在二进制文件里会明文出现 CreateRemoteThread 字符串。
- 杀毒软件和 EDR 在做静态扫描时，会直接搜索二进制文件中的敏感字符串。一旦发现 "CreateRemoteThread"、"WriteProcessMemory"、"VirtualAllocEx" 这类字符串同时出现，就基本可以断定这是个注入工具，直接拉黑。

所以我们需要一种方法：不写出函数名的明文，也能找到它的地址。

这就是用哈希值代替明文函数名，手工遍历 DLL 的导出表，找到目标函数。

● 导出表是什么（回顾及深入）

一个 DLL（比如 kernel32.dll）本身是个 PE 文件。PE 文件里有段区域，专门记录了“这个 DLL 对外提供了哪些函数”，这段区域就叫导出表（Export Table）。

导出表里最核心的是三张“清单”：

清单名字	它存了什么
名字数组(AddressOfNames)	每个导出函数的名字(如 "CreateFile"、"ReadFile")
序号数组(AddressOfNameOrdinals)	每个函数名对应的序号(用来和下面那张表对应)
地址数组(AddressOfFunctions)	每个导出函数的真实地址偏移量(RVA)

它们的关系是这样的：

名字数组[0] = "CreateFile" → 序号数组[0] = 5 → 地址数组[5] = 函数地址的 RVA

名字数组[1] = "ReadFile" → 序号数组[1] = 8 → 地址数组[8] = 函数地址的 RVA

物理意义：你拿着一个函数名，先在“名字数组”里找到它的位置（索引 i），然后用索引 i 去“序号数组”里取出一个数字，最后用这个数字去“地址数组”里取出真正的函数地址。

这就是 GetProcAddress 内部在做的事。我们现在要手工把它复现一遍。

● ROR13 哈希是什么

如果我们不想在代码里写明文 "CreateRemoteThread"，我们需要一种方法：把字符串转换成一个数字，然后只存这个数字。

这个“把字符串变成数字”的过程，就叫哈希。哈希算法有很多种，我们这里用最简单的一种：ROR13。

ROR13 的物理动作

ROR 是 Rotate Right（循环右移）的缩写。

- 普通右移 (>>)：把二进制位往右移，右边移出去的位直接丢掉。
- 循环右移：把二进制位往右移，右边移出去的位补到左边来。

比如一个 32 位整数 0x00000001：

- 普通右移 1 位：0x00000000（丢掉了 1）
- 循环右移 1 位：0x80000000（1 从右边跑到左边去了）

ROR13 的意思就是：循环右移 13 位。

ROR13 哈希的计算流程

- (一) 把哈希值初始化为 0。
- (二) 对字符串的每一个字符：
 - a) 把当前哈希值循环右移 13 位。
 - b) 加上当前字符的 ASCII 码。

(三) 字符串结束后，得到的数字就是哈希值。

以下便为其 Hash 的代码实现方式：

```
DWORD Ror13Hash(const char* str){
    DWORD hash{0};
    while(*str){
        hash = (hash >> 13) | (hash << (32-13));
        hash += (unsigned char)*str;
        str++;
    }
    return hash;
}
```

注意：这个过程是不可逆的。你只能从字符串算出哈希值，不能从哈希值反推出字符串。但因为它对不同的字符串产生的哈希值几乎不可能相同（碰撞概率低），所以我们可以用哈希值来唯一标识一个函数名。

● 完整的利用链

现在我们把所有东西串起来。

离线阶段（写代码之前）

- 算好 "CreateRemoteThread" 的 ROR13 哈希值（比如 0x5C0E1A2B）。
- 把这个数字硬编码到代码里。代码里不再出现任何明文函数名。

运行时阶段（程序执行时）

- 拿到 kernel32.dll 的基址。
- 从基址出发，沿着固定的偏移链，到达导出表。
- 遍历“名字数组”，对每一个函数名算 ROR13 哈希。
- 如果某函数哈希值等于硬编码的 0x5C0E1A2B，说明找到了 CreateRemoteThread。
- 用它的索引去“序号数组”和“地址数组”里取出函数的真实地址。
- 调用这个地址。

最终效果：二进制文件里，只有一堆十六进制数字，没有任何敏感字符串。